

```
['health', 4.0, inf, [['hhmb', -inf, 1.0, 4.948717948717949], ['hhmb', 1.0, inf, 6.132075471698113]]]]]]]
```

In particular, notice that the first branch is split on the variable `health` instead of the variable `netusoft`. Other branches at lower depths are split at different points and for different variables. The flexibility of the decision tree method means that starting with the same dataset and the same end goal, two researchers can potentially reach very different conclusions, depending on the parameters they use and decisions they make about how to work with the data. This is a common characteristic of machine learning methods, and part of what makes them so difficult to master.

Evaluating Our Decision Tree

In order to generate our decision tree, we compared error rates for each potential split point and each potential splitting variable, and we always chose the variable and split point that led to the lowest error rate for a particular branch. Now that we've successfully generated a decision tree, it makes sense to do similar error calculations, not just for a particular branch but for the whole tree. Evaluating the error rate for the whole tree can give us a sense of how well we've accomplished our prediction task, and how well we're likely to perform on future tasks (for example, future hospital patients complaining of chest pain).

If you look at the decision tree output that we've generated so far, you'll notice that it's a little hard to read all the nested lists, and there's no natural way to determine how happy we predict someone is without painstakingly reading through the nested branches and finding the right terminal node. It will be helpful for us to write code that can determine the predicted level of happiness for a person based on what we know about them from their ESS answers. The following function, `get_prediction()`, can accomplish this for us:

```
def get_prediction(observation, tree):
    j = 0
    keepgoing = True
    prediction = - 1
    while(keepgoing):
        j = j + 1
        variable_tocheck = tree[0][0]
```